

Practical - Week12

1. Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
GNU nano 8.7.1 producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

int *buffer;
int size, items;
int in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
long long checksum = 0;
int consumed = 0;
void *producer(void *arg) {
    for (int i = 1; i <= items; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        in = (in + 1) % size;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}
void *consumer(void *arg) {
    for (int i = 0; i < items; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        out = (out + 1) % size;
        checksum += item;
        consumed++;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    return NULL;
}
```

```

root@DESKTOP-QU0HVJI:~/practical/practical-12# nano producer_consumer.c
root@DESKTOP-QU0HVJI:~/practical/practical-12# gcc -pthread producer_consumer.c -o pc
root@DESKTOP-QU0HVJI:~/practical/practical-12# ./pc 1 1000000
./pc 10 1000000
./pc 100 1000000
./pc 1000 1000000
Buffer size: 1
Items produced: 1000000
Items consumed: 1000000
Expected checksum: 500000500000
Actual checksum: 500000500000
Execution time: 59.757650 seconds
Throughput: 16734.259128 items/second
Buffer size: 10
Items produced: 1000000
Items consumed: 1000000
Expected checksum: 500000500000
Actual checksum: 500000500000
Execution time: 6.682136 seconds
Throughput: 149652.745769 items/second
Buffer size: 100
Items produced: 1000000
Items consumed: 1000000
Expected checksum: 500000500000
Actual checksum: 500000500000
Execution time: 0.928663 seconds
Throughput: 1076816.886212 items/second
Buffer size: 1000
Items produced: 1000000
Items consumed: 1000000
Expected checksum: 500000500000
Actual checksum: 500000500000
Execution time: 0.770358 seconds
Throughput: 1298097.767531 items/second
root@DESKTOP-QU0HVJI:~/practical/practical-12# |

```

Report: The Producer-Consumer problem was implemented using POSIX threads and counting semaphores. Correct synchronization was verified by matching the expected and actual checksums, while throughput was measured for different buffer sizes.

2. Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```

GNU nano 8.7.1                                deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t R1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t R2 = PTHREAD_MUTEX_INITIALIZER;
pthread_barrier_t barrier;
void *thread1(void *arg) {
    printf("Thread 1: Requesting Resource 1\n");
    pthread_mutex_lock(&R1);
    printf("Thread 1: Locked Resource 1\n");
    pthread_barrier_wait(&barrier);
    printf("Thread 1: Requesting Resource 2\n");
    pthread_mutex_lock(&R2);
    printf("Thread 1: Acquired both resources\n");
    pthread_mutex_unlock(&R2);
    pthread_mutex_unlock(&R1);
    return NULL;
}
void *thread2(void *arg) {
    printf("Thread 2: Requesting Resource 2\n");
    pthread_mutex_lock(&R2);
    printf("Thread 2: Locked Resource 2\n");
    pthread_barrier_wait(&barrier);
    printf("Thread 2: Requesting Resource 1\n");
    pthread_mutex_lock(&R1);
    printf("Thread 2: Acquired both resources\n");
    pthread_mutex_unlock(&R1);
    pthread_mutex_unlock(&R2);
    return NULL;
}
int main() {
    pthread_t t1, t2;
    setbuf(stdout, NULL);
    pthread_barrier_init(&barrier, NULL, 2);
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);
}

root@DESKTOP-QU0HVJI:~/pratical/practical-12# nano deadlack.c
root@DESKTOP-QU0HVJI:~/pratical/practical-12# gcc -pthread deadlack.c -o deadlack
root@DESKTOP-QU0HVJI:~/pratical/practical-12# timeout 5s ./deadlack
Thread 1: Requesting Resource 1
Thread 1: Locked Resource 1
Thread 2: Requesting Resource 2
Thread 2: Locked Resource 2
Thread 2: Requesting Resource 1
Thread 1: Requesting Resource 2
root@DESKTOP-QU0HVJI:~/pratical/practical-12# |

```

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t R1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t R2 = PTHREAD_MUTEX_INITIALIZER;

void *work(void *arg) {
    int id = *(int *)arg;
    printf("Thread %d requesting Resource 1\n", id);
    pthread_mutex_lock(&R1);
    printf("Thread %d acquired Resource 1\n", id);
    pthread_mutex_lock(&R2);
    printf("Thread %d acquired Resource 2\n", id);
    printf("Thread %d using both resources\n", id);
    pthread_mutex_unlock(&R2);
    pthread_mutex_unlock(&R1);
    printf("Thread %d released resources\n", id);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int a = 1, b = 2;
    pthread_create(&t1, NULL, work, &a);
    pthread_create(&t2, NULL, work, &b);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_mutex_destroy(&R1);
    pthread_mutex_destroy(&R2);
    printf("No deadlock occurred\n");
    return 0;
}
```

```
Thread 1 requesting Resource 1
root@DESKTOP-QU0HVJI:~/practical/practical-12# nano ordered.c
root@DESKTOP-QU0HVJI:~/practical/practical-12# gcc -pthread ordered.c -o ordered
root@DESKTOP-QU0HVJI:~/practical/practical-12# ./ordered
Thread 1 requesting Resource 1
Thread 1 acquired Resource 1
Thread 2 requesting Resource 1
Thread 1 acquired Resource 2
Thread 1 using both resources
Thread 1 released resources
Thread 2 acquired Resource 1
Thread 2 acquired Resource 2
Thread 2 using both resources
Thread 2 released resources
No deadlock occurred
root@DESKTOP-QU0HVJI:~/practical/practical-12# |
```

Report: The ./deadlock program demonstrated deadlock using pthread_mutex_lock(). The ./ordered program used resource ordering to prevent circular wait, allowing both threads to complete successfully.